

MarketRadar — Demo Video Guide

Scene-by-scene recording plan and test-data usage · October 2026

A complete, step-by-step plan for recording the submission video. It covers what to prepare, what to click, what to say, and **which test data to use in each scene**.

Target length: **10–12 minutes**. Scene timings are guides only.

Part 1 — Understand the three kinds of test data

Using the right data in each scene is what makes the demo smooth.

Data	Where it lives	Costs credits?	Use it to show
A. Live searches	your real database, backend/data/market_radar.db	Yes, the first time; repeats within 7 days are free (cached)	Competitors, Products, Price Intelligence, Market Research working on real data
B. Demo history	separate file, backend/data/demo_market_radar.db , created by scripts/seed_demo_data.py	No	Trends with real directions , Market Pulse statistics, a full AI Analyst report
C. Automated test data	inside backend/tests/ and built by pytest	No	That the system is tested: 525 tests, no network, isolated database

Why you need B: a trend requires two *distinct* measurements. Repeating a search inside the 7-day cache window returns the same response, so real data gathered this week will honestly show "**insufficient history**". The demo file contains two searches 10 days apart, so directions appear. Its businesses use `.example` domains and its location is literally "**Sample City (demo data)**". Say on camera that it is sample data.

Part 2 — Preparation (do this the day before)

2.1 Install and verify

Follow README section 4 once, end to end. Then confirm:

```
cd backend; .\venv\Scripts\Activate.ps1; pytest           # expect: 525 passed
cd ..\frontend; npm run build                             # expect: built, no errors
```

2.2 Rehearse the live searches (this is what makes the recording free)

1. Start the backend normally (real database) and the frontend.

2. Run every live search from the table below **exactly as written**. This spends credits once and fills the 7-day cache.
3. Record the video **within 7 days**. The same inputs are then cache hits: instant, identical and free. The results show a **"Cached result"** chip, which you can mention.

Scene	Page	Inputs (type exactly)	Approx. credits on rehearsal
4	Market Research	Query Wine stores , Location Holmdel, NJ	1
5	Competitors	Market Wine Retail , Location Holmdel, NJ	1
6	Products	Product Kendall Jackson Vintner's Reserve Chardonnay 750ml , Market Wine Retail , Location Holmdel, NJ	1 + up to 5
7	Price Intelligence	Product Cycling Frog 5mg Grapefruit 12oz 6pk , Location 08807 , Radius 25	≈3 + 1 per nearby store with a website

Budget about **15–25 credits** for the rehearsal. Check your remaining quota at <https://serpapi.com/dashboard> before you start.

Inputs must match **character for character**, including market, location and radius, or the cache misses and the search is live again.

2.3 Create the demo history

```
cd backend
python scripts\seed_demo_data.py          # add --reset to rebuild
```

2.4 Screen setup

- Browser at 100–110% zoom, window about 1440×900, sidebar visible.
- Close other tabs. Turn notifications off.
- Open a second browser tab at `http://localhost:8001/docs` (Swagger) for the API scene.
- Keep a terminal ready in `backend/` with the venv active, for the tests scene.
- Have one architecture slide ready (the diagram from section 5.1 of the Project Document), or show that section of the document.

Part 3 — Scene-by-scene script

Each scene lists **Do** (actions), **Say** (suggested narration — adapt freely) and **Data** (which test data is on screen).

Scene 1 — Introduction (0:00–0:45)

Do: show the Dashboard (`http://localhost:5173`).

Say: "This is MarketRadar, a market-intelligence platform for new and growing businesses. You give it a market, a location and optionally a product. It finds the real competitors, checks what they sell and at what verified price, tracks what changes, and summarises it as facts with evidence. One rule runs through the whole system: it reports what was observed and how strongly it is evidenced, and it never invents a conclusion."

Data: none.

Scene 2 — The problem (0:45–1:30)

Do: stay on the Dashboard or show a title slide.

Say: "Search results mix real businesses with directories, blog articles, marketplaces and social pages. A price on a category page isn't the product's price. A cached result read twice looks like 'no change' when nothing was measured twice. And AI summaries happily invent market leaders and growth. MarketRadar is designed around each of these traps."

Scene 3 — Architecture (1:30–2:30)

Do: show the architecture diagram.

Say:

- "The frontend is React and TypeScript. The backend is FastAPI with SQLite. Market data comes from SerpApi."
- "There are two kinds of module. Discovery modules — Market Research, Competitors, Products and Price Intelligence — call SerpApi through one component that handles caching, credit accounting and error semantics."
- "The read-only modules — Trends, Market Pulse and the AI Analyst — only read stored observations. They never call the provider, never spend a credit and never write."
- "Businesses are stored once as merchants. Every search appends observations as evidence, so history is never overwritten."

Scene 4 — Market Research (2:30–3:15)

Data: A — live (cached). Query `wine stores`, Location `Holmdel, NJ`.

Do:

1. Open **Market Research**.
2. Type the location and query, then click **Search**.
3. Point at **"resolved to Holmdel, New Jersey, United States"** and the **Cached result** chip.
4. Switch between the Organic and Local tabs.

Say: "I typed 'Holmdel, NJ'. SerpApi only accepts its own catalogue names, so MarketRadar resolves the location first using SerpApi's free location catalogue. This result came from the 7-day cache, so it cost nothing."

Scene 5 — Competitors (3:15–4:45)

Data: A — live (cached). Market `wine Retail`, Location `Holmdel, NJ`.

Do:

1. Open **Competitors** and run the search.
2. Show the funnel counts (local, organic, rejected, verified, discovered).
3. Scroll to the **rejected results**: directories and articles, each with a reason.
4. Open one competitor's **evidence** dialog.
5. Point out a **discovered** business with "**Location not verified**".

Say:

- "Local map results are structured businesses with addresses, so they're verified."
- "Organic results are classified by the shape of the page. These were rejected as directories and articles, and the reason is kept."
- "This one is only *discovered*. It came from a single organic result with no address, so the system says it couldn't verify the location rather than pretending it's local."

Scene 6 — Products (4:45–6:00)

Data: A — live (cached). Product `Kendall Jackson Vintner's Reserve Chardonnay 750ml`, Market Wine Retail, Location `Holmdel, NJ`.

Do:

1. Open **Products** and run the search.
2. Show a row that is **found + verified price (19.99)** and rows that are **unknown**.
3. Hover over the reason and the evidence-scope chip (**Catalogue**).
4. Show the rejected rows.

Say:

- "For each competitor, MarketRadar searches that business's own website for the exact product. Product status and price status are separate."
- "This store's site shows the exact bottle at 19.99, so the price is verified."
- "These are *unknown* — not 'not sold'. No evidence either way is not proof of absence."
- "A catalogue page proves the merchant lists the product, not that a store has it on the shelf, so inventory is never claimed."
- "Identity is strict: a different size, a 'Reserve' variant or an accessory doesn't match."

Scene 7 — Price Intelligence (6:00–7:00)

Data: A — live (cached). Product `Cycling Frog 5mg Grapefruit 12oz 6pk`, Location `08807`, Radius `25 miles`.

Do:

1. Open **Price Intelligence** and run the search.
2. Point at the "**Results for ... within 25 miles of ...**" line.
3. Show the statistics card.
4. Show the nearby-stores table and its status column.
5. Hover over a "**Price unavailable**" info icon.

Say: "This finds physical stores around the area within the radius and gathers evidence from their websites and Google Shopping. Only verified prices count towards lowest, average and highest. With fewer than two distinct prices, it lists the price instead of pretending there's an average."

If the result has no verified price, say: "Here the products were found, but no price could be tied to the exact product. The page says that rather than showing a number."

Scene 8 — Validation, and errors that cost nothing (7:00–7:45)

Data: inputs that are rejected before any search (free).

Do on Competitors:

1. Market `Coffee Shops`, Location `Holmdel, Germany` → refused: "'Germany' does not match the closest supported location..."
2. Location `asdfqwerzx` → "could not be matched to a supported search location".
3. Market `x`, any location → "too short to describe a market".

Say: "Bad input is refused before any credit is spent. Notice 'Holmdel, Germany' — a careless system would quietly search Holmdel, New Jersey. MarketRadar refuses and explains why."

Scene 9 — Switch to demo history (7:45–8:00)

Do:

1. In the backend terminal, press Ctrl+C, then run:

```
$env:DATABASE_URL="sqlite:///./data/demo_market_radar.db"
uvicorn app.main:app --reload --port 8001
```

2. Refresh the browser.

You can also record Scenes 10–12 as a separate clip and join them in editing.

Say: "Trends need two real measurements taken at different times. Live searches I repeat this week come from the cache, so they honestly show 'insufficient history'. To show trends, I've switched to a separate demo database of clearly labelled sample data. Its location is literally 'Sample City (demo data)', and it contains two searches ten days apart."

Scene 10 — Trends (8:00–9:00)

Data: B — demo history.

Do:

1. Open **Trends**.
2. Show the summary counts.
3. **Prices** tab: House Blend price **14.99** → **15.99, up**. The second store, at 13.49 both times, shows **insufficient history**.
4. **Visibility** tab: Example Roasters' position **2** → **1**, shown as an improvement.
5. **Merchant presence** tab: **Demo Corner Kiosk** — "**not in latest**". Hover over the tooltip.

Say:

- "Here one price rose between two distinct measurements."

- "This store read 13.49 both times. MarketRadar doesn't call that 'stable', because an identical reading could just be the cache read twice. It stays 'insufficient history'."
- "This business wasn't in the latest search. That's reported as 'not in latest', never as 'closed'."

Scene 11 — Market Pulse (9:00–9:45)

Data: B — choose **Coffee Shops — Sample City (demo data)**.

Do:

1. Open **Market Pulse**, pick the context, and click **Load snapshot**.
2. Show the data-quality chips and the **Prices** tab: statistics across **3 distinct prices** (13.49–15.99, average 14.82).
3. Show the **Trends** tab.
4. Briefly select **Bike Repair** to show every series with insufficient history.

Say: "Market Pulse is one snapshot of one market and place. Every section is filtered to the same context, so one market's competitors can never appear next to another market's prices. It's read-only and costs nothing."

Scene 12 — AI Analyst (9:45–11:00)

Data: B — **Coffee Shops — Sample City (demo data)**.

Do:

1. Open **AI Analyst**, pick the context, and click **Analyze**.
2. Point at the **"Facts only (deterministic)"** chip.
3. Scroll through the sections, highlighting the trend statement ("increased from 14.99 to 15.99") and an evidence chip.

Say:

- "The AI is not the source of truth. Every statement here is computed by code from the snapshot, and each one carries a reference to the records behind it."
- "A language model can optionally add a short narrative. It only ever sees these computed statements. Its text is then checked, and rejected if it claims a market leader, growth, a prediction or a recommendation, or uses any number that isn't in the facts."
- "No AI vendor is configured here, so it runs in deterministic mode — and the full analysis is still available."

Scene 13 — Quality and tests (11:00–11:45)

Data: C — automated test data.

Do:

1. In the terminal (`backend/` , `venv` active), run `pytest` and show **525 passed**.
2. Optionally show `backend/tests/test_audit_regressions.py` in the editor.
3. Optionally show Swagger at `http://localhost:8001/docs` .

Say: "There are 525 automated tests. They run against a throwaway database, every provider call is stubbed, and the suite spends no credits. There's a regression test for every defect found in a pre-submission audit."

Scene 14 — Limitations and close (11:45–12:30)

Do: show the Dashboard, or the limitations section of the README.

Say: "To be transparent about the limits: prices are parsed in dollars, product matching is token-based, organic results carry no geography, and no AI vendor adapter is bundled yet. Next on the roadmap are an AI adapter, PostgreSQL and multi-currency support. MarketRadar's value is that what it tells you is backed by evidence, and what it can't know, it says it can't know. Thank you."

Part 4 — Say this, not that

Say	Don't say
"verified price"	"the market price" / "the cheapest price in town"
"not in the latest search"	"closed" / "went out of business"
"insufficient history"	"stable" / "no change"
"listed in the merchant's online catalogue"	"in stock"
"position 1 in that search"	"the market leader" / "the best competitor"
"the analyst states observed facts"	"the AI predicts / recommends"
"sample data" (for the demo file)	presenting demo businesses as real

Part 5 — If something goes wrong while recording

Problem	What to do
A live search shows "The search provider did not respond" (502)	Retry once. If it fails again, say "the provider failed — note it's reported as a failure, not as 'no results'", and move on. That honesty is itself a feature.
A search is unexpectedly slow	The cache missed: an input differs from the rehearsal, or 7 days have passed. Let it finish or cut in editing.
"SerpApi key not configured"	The backend started without <code>backend/.env</code> . Fix it and restart.
Market Pulse / Analyst list is empty	The backend is on the wrong database. Check <code>DATABASE_URL</code> and restart.
Trends show only "insufficient history"	You are on the real database. Switch to the demo database (Scene 9).
Sidebar says Backend Unavailable	The backend is not running on port 8001.

Part 6 — After recording

1. Stop the backend.
2. Close the terminal, or run `Remove-Item Env:DATABASE_URL`, so the next start uses the real database again.

3. The demo file `backend/data/demo_market_radar.db` can be kept or deleted. It is never read unless `DATABASE_URL` points to it.
 4. Package the submission as described in README section 12. Exclude `backend/.env`, `venv/`, `node_modules/`, `*.db` and `_quarantine/`.
-

Part 7 — One-page checklist

- ☐ `pytest` → 525 passed; `npm run build` passes
- ☐ SerpApi quota checked (≈ 25 credits free)
- ☐ Rehearsal searches done within the last 7 days, using the exact inputs in Part 2.2
- ☐ Demo database created (`python scripts/seed_demo_data.py`)
- ☐ Browser zoom set, notifications off, Swagger tab open
- ☐ Architecture slide ready
- ☐ Terminal open in `backend/` with `venv` active
- ☐ Scenes 1–8 on the real database, Scenes 9–12 on the demo database, Scene 13 tests
- ☐ Demo data called "sample data" on camera
- ☐ `DATABASE_URL` reset after recording; `.env` excluded from the submission